Team Collaboration SOP

How our software team works together, shares documents, and prepares for client meetings

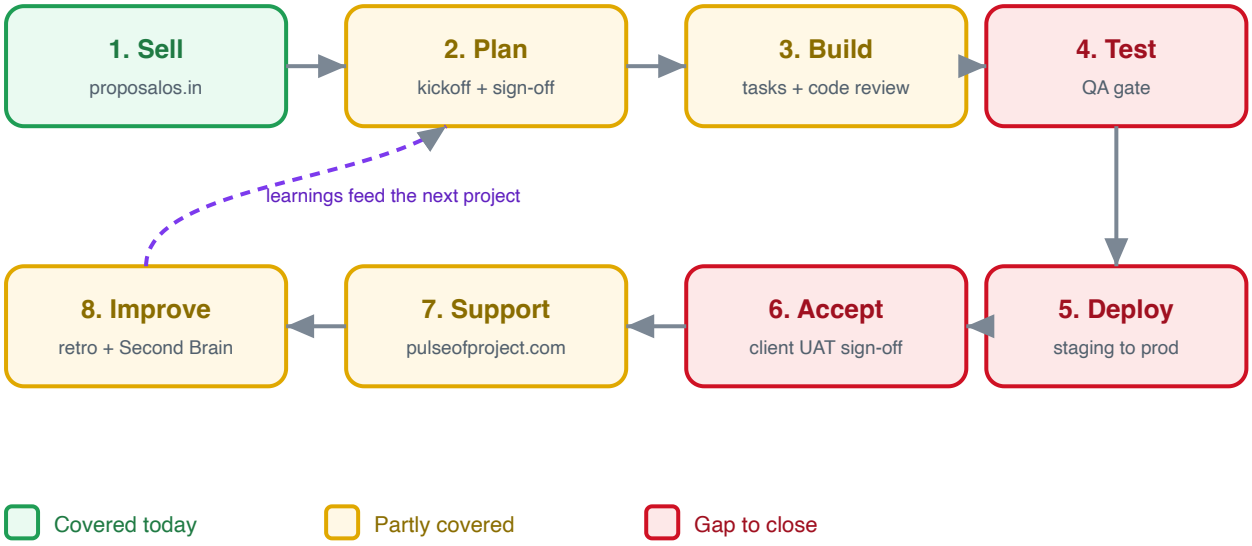
Standard Operating Procedure

HopeTech / Bettroi software team

Version 2.2 · June 2026

The full delivery lifecycle

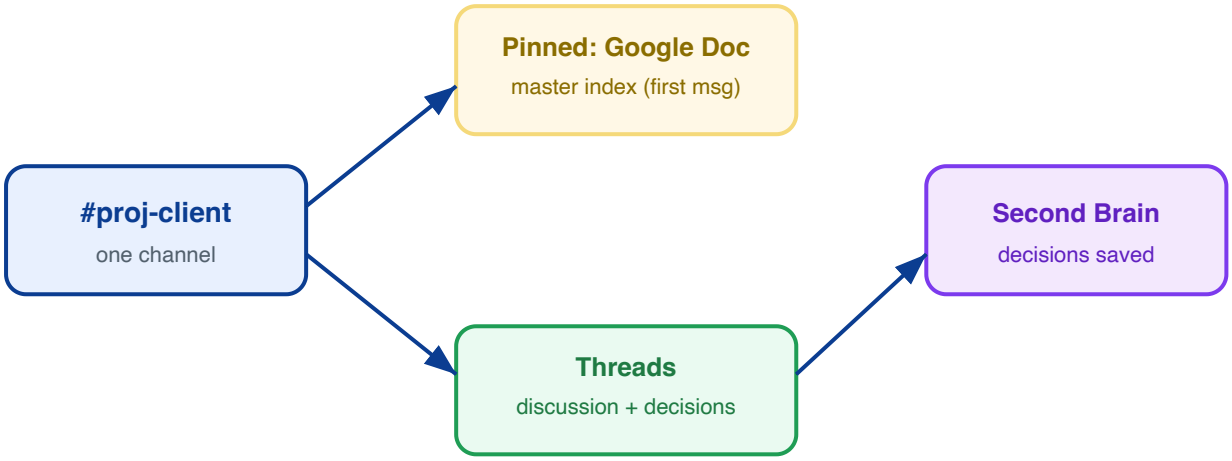
Every project moves through eight stages. Green stages we already run well; amber are partly covered; red are the gaps this SOP now closes.



Pages 1 to 5 cover how we communicate and keep documents straight (the foundation). Pages 6 to 12 add the delivery steps that turn this into a complete software workflow.

1 Slack is our single collaboration channel

All project conversation lives in Slack. Every project gets one channel, and its first pinned message is the Google Doc index.



One channel per project, named **#proj-client**. No scattered DMs for project work.

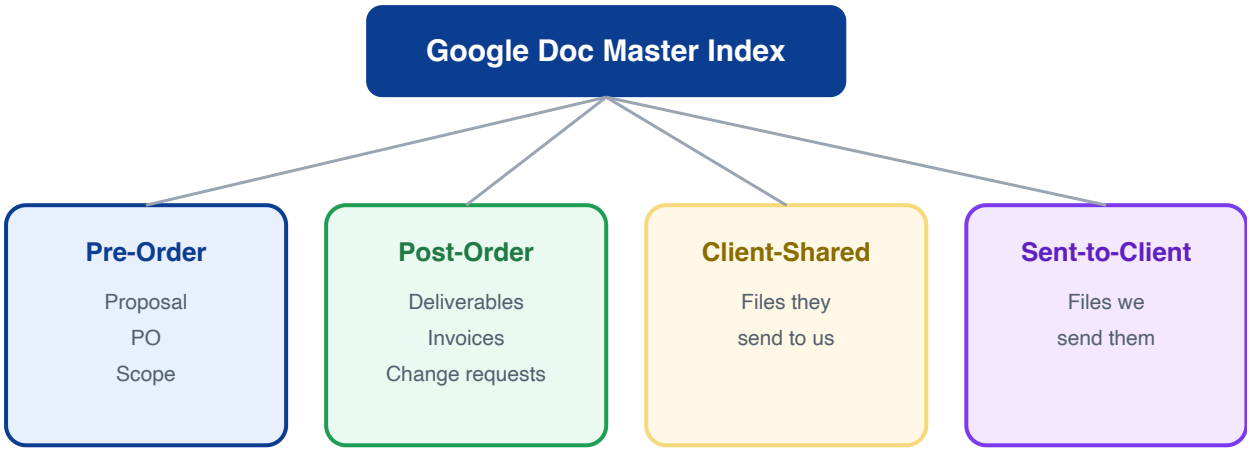
First pinned message = the Google Doc master index (see page 2). Always one click away.

Use threads for discussions. Key decisions get summarized back into the Second Brain.

External collaborators join via Slack Connect, never by sharing internal channels.

2 The Google Doc master index

One living Google Doc per client holds every document link, before and after the order. It is the map to everything.



> Every shared file gets a **row in the index** AND a copy/link in the client's vault folder.

> The index URL is the **pinned first Slack message** and is also recorded in the Second Brain.

> **Source of truth = the index + the vault**, never someone's local Downloads folder.

3 The 4:00 PM pre-demo standup

One day before any demo or client meeting, the whole team meets at 4:00 PM to confirm we are ready.

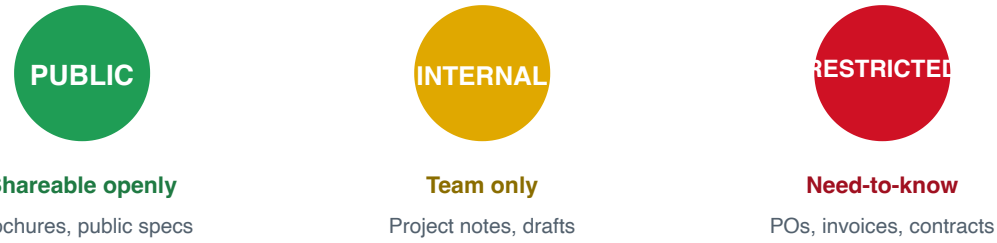
Readiness checklist (run every time):

- Demo script ready and rehearsed
- Environment and test data verified working
- Open bugs triaged; blockers flagged
- An owner assigned to each talking point
- Fallback plan if something breaks live

The standup is auto-captured: it appears in the calendar feed, and if recorded, as a meeting note in the project folder.

4 File and document access tiers

Every document carries a sensitivity tier. This decides where it can be shared and stored.



Shareable openly
Brochures, public specs

Team only
Project notes, drafts

Need-to-know
POs, invoices, contracts

Public — freely shareable material.

Internal — project notes and drafts; team only, never in public channels.

Restricted — anything with pricing, money, or PII: POs, invoices, contracts.

PHI is never committed to the shared vault. It stays in the gitignored scratch space.

Naming: `YYYY-MM-DD - client - doc-type - short-title`

5 Bug tracking and who does what

Bug tracking and the who-does-what view live in **pulseofproject.com**, which is connected to the Second Brain vault by API. Each bug links back to its Slack thread and its row in the Google Doc index.

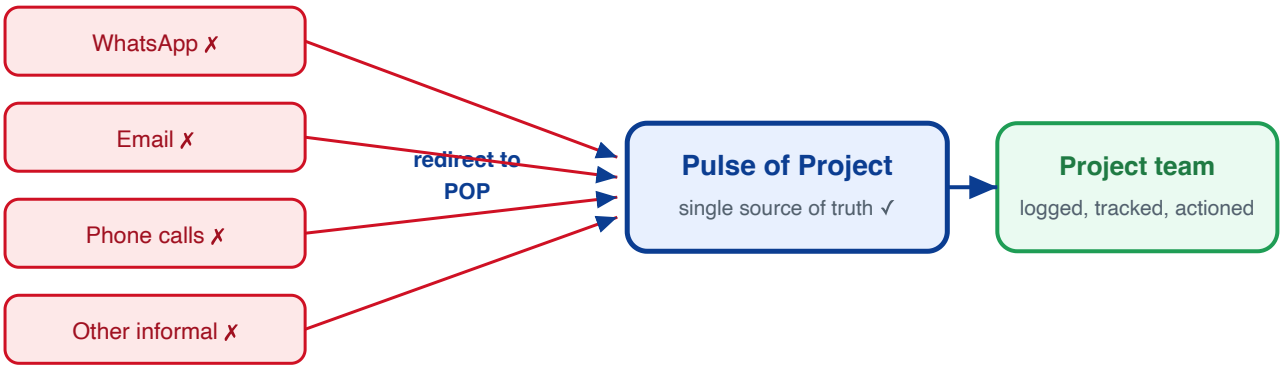
Activity	Owner	Where it lives
Project conversation	Whole team	Slack #proj-client
Document links (pre and post order)	Project lead	Google Doc master index
Pre-demo readiness	Whole team @ 4 PM	Standup + checklist
Client bugs and who-does-what	Assigned dev	pulseofproject.com (API to vault)
Restricted docs (PO, invoice)	Project lead	Restricted tier, not public

Source of truth: the Google Doc index plus the Second Brain vault. pulseofproject.com syncs bug and task status into the vault by API.

KEY POLICY

All client feedback goes through Pulse of Project (POP)

POP is the official and **only** channel for the client to log feedback. This is what makes the project trackable.



POP is the only channel for the client to submit feedback on deliverables, milestone submissions, UAT, bug reports, change requests (CRs), and enhancement requests.

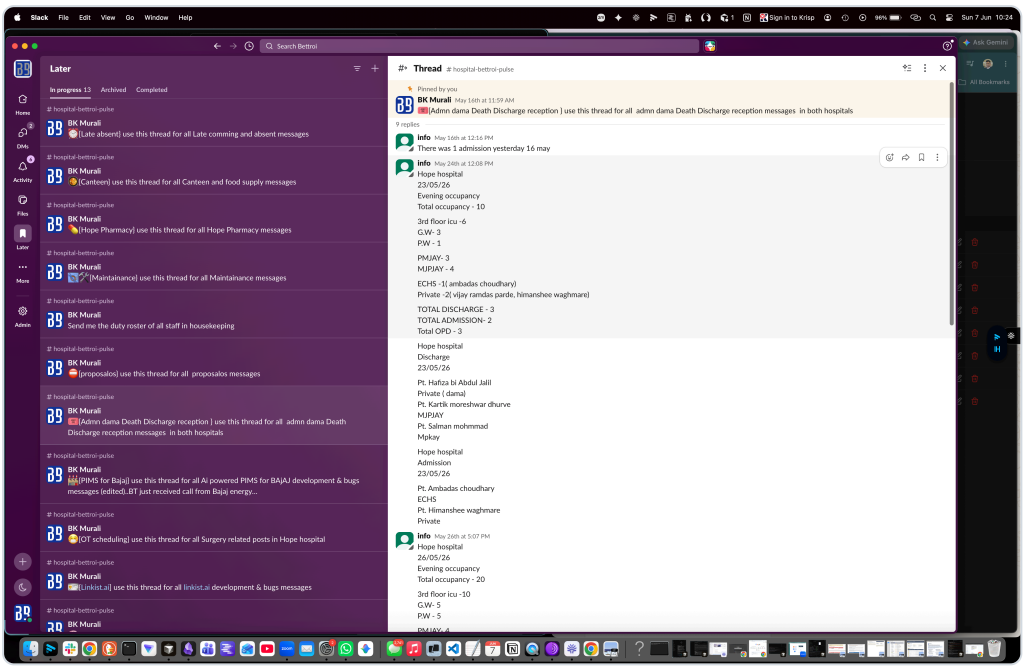
Feedback that arrives via **WhatsApp, email, phone, or any informal channel is redirected to POP** by the team, so it is documented and tracked.

If it is not in POP, it is **not on the official list** the team works from.

Why this matters: a lot of productive time is currently lost extracting feedback from scattered channels and consolidating it for the team. Making POP the single source of truth improves **accountability, traceability, and overall efficiency**. POP enhancements to better support this process are in progress (discussed with Sahil).

Slack thread directory — internal dev coordination

Each project has one dedicated Slack thread for development & bugs, bookmarked in **Later** so it is always one click away. One thread per topic keeps internal coordination from scattering.



Our Slack **Later** view: one bookmarked thread per topic, each pinned "use this thread for all … messages".

Thread	Use it for
Bajaj PIMS — dev & bugs	All AI-powered PIMS for Bajaj development & bug messages
Hope — dev & bugs	All Hope product development & bug messages
<new project> — dev & bugs	Pattern: one "development & bugs" thread per project, pinned "use this thread for all <project> development & bugs messages"

> **One thread per project** for dev & bug coordination; bookmark it in Slack **Later**.

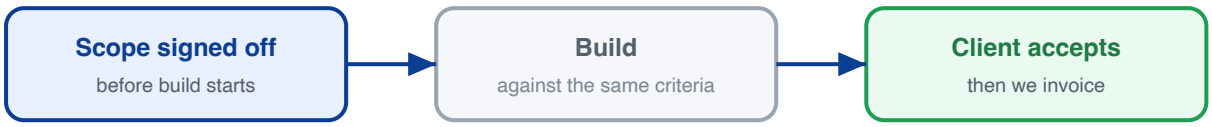
> Pin the thread with: *"use this thread for all <project> development & bugs messages"*.

> These threads are for **internal team coordination**. Official client feedback still goes through **Pulse of Project (POP)** — see the POP policy.

Note: this directory was set up recently as part of our internal collaboration. Keep it current — add a new dev & bugs thread whenever a new project starts.

6 Requirements and acceptance sign-off

Two written gates wrap every order: the client agrees the scope before we build, and confirms it works before we invoice. Same checklist, checked twice.



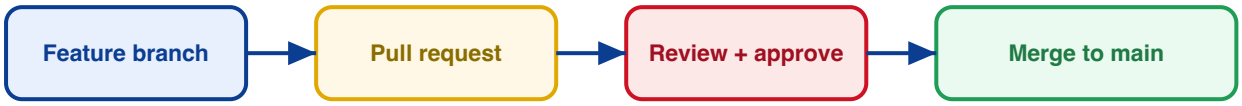
Acceptance criteria are written into the scope doc (in proposalos.in / Google Doc index) and the client signs it before work starts.

Change requests go through the same sign-off, never verbally; each is logged in the Google Doc index.

Acceptance test at delivery: client confirms each criterion is met. That sign-off triggers the invoice.

7 Code review and Git workflow

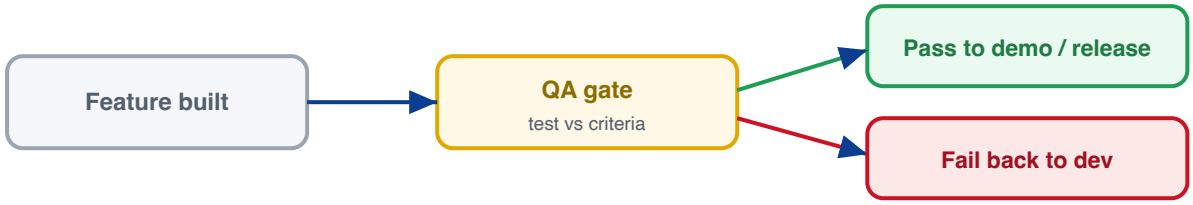
No code reaches the main branch without review. This is where most bugs are caught before a client ever sees them.



- **Branch per task**; never commit straight to main.
- **Pull request reviewed by at least one other developer** before merge.
- **No hardcoded secrets**; credentials come from environment variables only.
- Each PR links back to its **pulseofproject.com** task.

8 Testing and the QA gate

A feature is tested against its acceptance criteria before any demo or release. Testing must not happen live in front of the client.



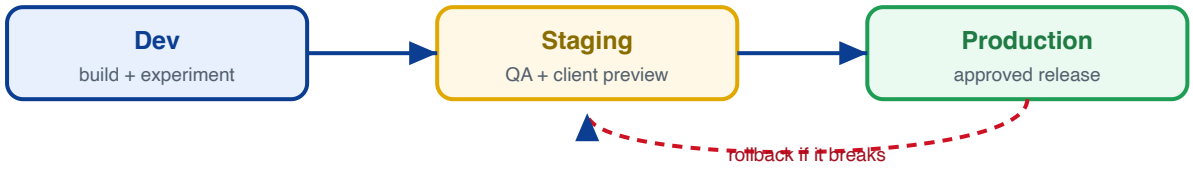
Someone other than the author **tests against the acceptance criteria**.

Bugs found are logged in **pulseofproject.com** and fixed before the gate is passed.

The **4 PM pre-demo standup** confirms the QA gate is green for everything being shown.

9 Deploy, environments and rollback

Changes flow through fixed environments. Every release has an approver and a way to undo it.



> **Dev to Staging to Production**; never edit production directly.

> Client-facing releases need a **named approver**.

> Every release has a **rollback plan** so a bad deploy can be reverted fast.

10 Incidents, support and escalation

When something breaks for a client in production, response time depends on how serious it is.

Severity	Example	Response
P1 Critical	System down, client cannot work	Immediate; all-hands hotfix
P2 High	Major feature broken, workaround exists	Same business day
P3 Normal	Minor bug, low impact	Next planned cycle

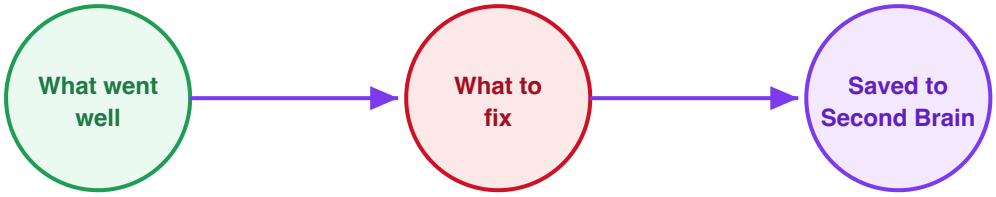
> All incidents are logged in **pulseofproject.com** and announced in the project Slack channel.

> P1 fixes follow the same **review and rollback** rules; speed never skips review.

> Each incident gets a one-line **root-cause note** saved to the Second Brain.

11 Retrospective and continuous improvement

After each project or major milestone, a short look-back so the next project runs better.



> Short retro after each project or milestone: **what went well, what to fix, one action each**.

> Lessons are saved to the **Second Brain** (and harvested as reusable patterns) so they compound.

12 Cross-cutting essentials

Three rules that apply across every stage above.

Access and secrets — new team members are granted repo / server / client-system access on joining and have it **revoked when they leave**. Credentials live in a secrets manager, never in chat or code.

Time and effort tracking — work is logged against the order so we can **bill accurately against the PO** in proposalos.in.

Definition of Ready / Definition of Done — a task only starts when it has clear acceptance criteria (Ready), and is only closed when built, reviewed, tested, and deployed (Done).

The one-line summary: Communicate in Slack, keep every link in the Google Doc index, sign off scope before and after, review and test before you ship, deploy safely, and write down what you learned.